

Diskrétní struktury

Část 3 Teorie grafů (5)



Vysoká škola
polytechnická
Jihlava

Minimální kostry

V základních algoritmech prohledávání grafu je výsledné pořadí objevení uzlů náhodné - záleží na pořadí probírání uzlů a na pořadí jejich uvedení v seznamech sousedů.

Někdy je ale potřeba vybírat podle nějakého kritéria optimality.

Určení minimální kostry

- užitečné, pokud má graf ohodnocené hrany (např. délkou)
 - minimální kostra má součet ohodnocení minimální, tj. neexistuje jiná kostra, která by měla menší celkovou délku
 - minimálních koster může být více
- vyřešeno českými matematiky Borůvkou a Jarníkem ve 20. a 30. letech - návrh sítě pro rozvod elektřiny na Moravě

Minimální kostry

Jarníkův-Primův algoritmus

- uvažujeme graf G , jehož hrany jsou ohodnoceny kladnými celými čísly
- postupně tvoříme kostru přidáváním „optimální“ hrany
- výběr hrany pro přidání do kostry
 - realizace prioritní frontou
 - ve frontě leží uzly, nezařazené zatím do kostry
 - ve frontě jsou uzly seřazeny podle své minimální vzdálenosti od dosud vytvořené kostry, $d[u]$
- na začátku stanovíme kořen r , z něhož budeme začínat
- vytváříme pole předchůdců $p[]$ jako v BFS

Minimální kostry

Jarníkuv-Primův algoritmus

Algoritmus 5.14 Určení minimální kostry grafu (Jarník – Prim)

JP-MST(G, w, r)

1 $Q := U$

2 **for** každý uzel $u \in Q$

3 **do** $d[u] := \infty$

4 $d[r] := 0$; $p[r] := \text{NIL}$

5 **while** $Q \neq \emptyset$ **do**

6 $u := \text{EXTRACT-MIN}(Q)$

7 **for** každý uzel $v \in \text{Adj}[u]$ **do**

8 **if** $(v \in Q)$ **and** $(w(u, v) < d[v])$

9 **then** $p[v] := u$

10 $d[v] := w(u, v)$

Do fronty ulož všechny uzly
a urči jim nekonečnou
vzdálenost od podstromu.

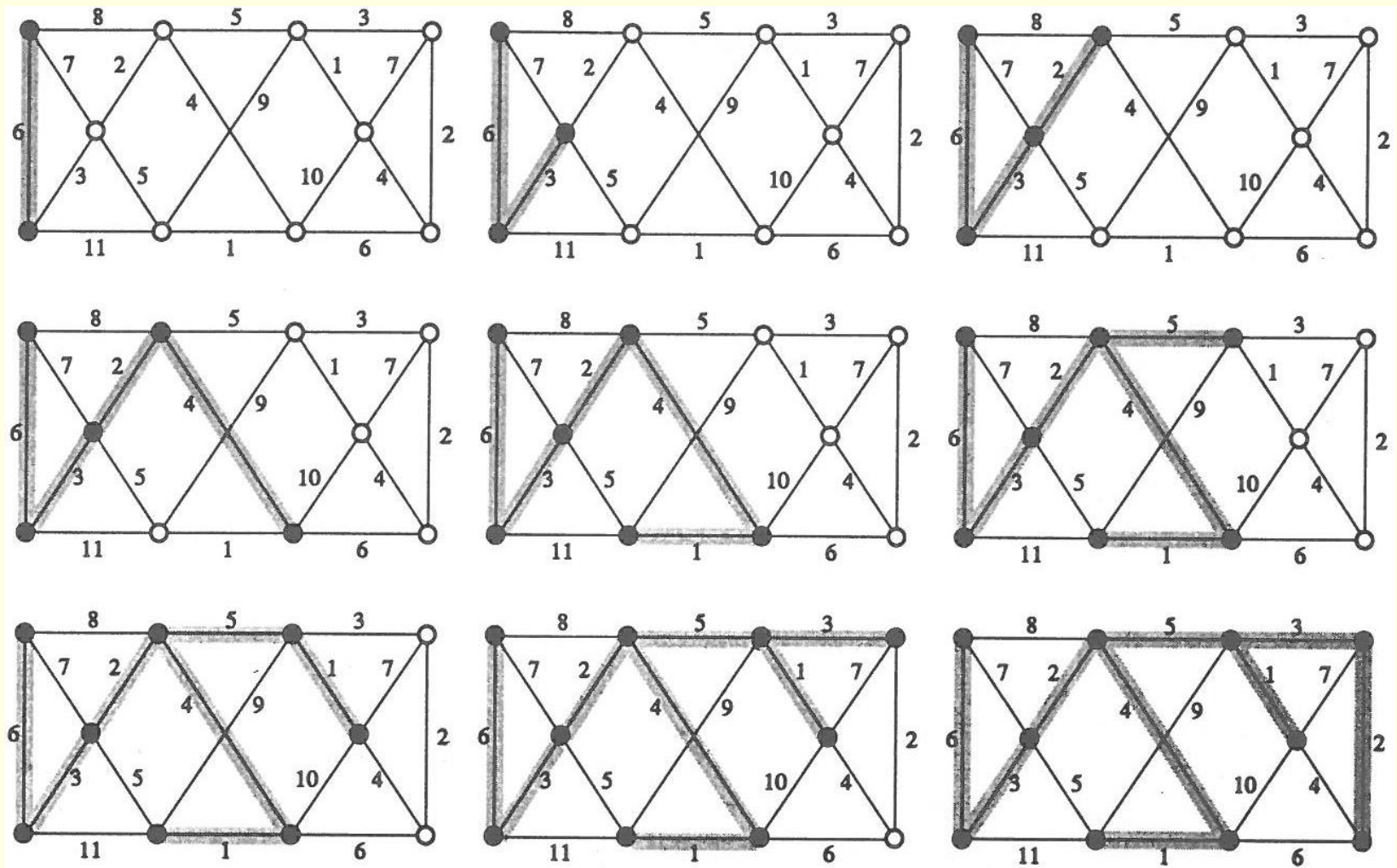
Pouze kořen nemá předchůdce.

Dokud není fronta prázdná,
vyber nejbližší uzel a sousedům z Q
zkus, zda mají blíže k u .

Pokud ano, zadej jim předchůdce u
a menší vzdálenost.

Minimální kostry

Jarníkuv-Primův algoritmus



Nejkratší cesty z jednoho uzlu

Vzdálenost na orientovaných a ohodnocených grafech

Jak to bude s vlastnostmi, které jsme zavedli u neorientovaných grafů?

(0) vzdálenost $d(u, v)$ je celé nezáporné číslo

Tuto vlastnost lze zachovat jen při nezáporném celočíselném ohodnocení všech hran.

(1) $d(u, v) \geq 0$; přitom $d(u, v) = 0$, právě když $u = v$

Tuto vlastnost lze zachovat jen při kladném ohodnocení všech hran.

(2) $d(u, v) = d(v, u)$

Se symetrií je třeba se u orientovaných grafů definitivně rozloučit.

(3) $d(u, v) \leq d(u, z) + d(z, v)$

Trojúhelníková nerovnost platí beze změny i pro w -vzdálenosti, neboť je dána podmínkou minimality a součtovým charakterem určení w -délky cesty.

(4) je-li $d(u, v) > 1$, pak existuje uzel $z \in U$ tak, že $u \neq z \neq v$ a platí $d(u, v) = d(u, z) + d(z, v)$

Pro w -vzdálenost je možné vyslovit analogické tvrzení, je jen třeba upravit jeho předpoklad. Podmínka $d_w(u, v) > 1$ totiž nevyjadřuje to, co je pro důkaz zapotřebí, tedy existenci nějakého vnitřního uzlu na minimální cestě.

Nejkratší cesty z jednoho uzlu

Vzdálenost na orientovaných a ohodnocených grafech

- základní úlohou při určování vzdáleností v orientovaném grafu G je nalezení nejkratších cest z jednoho zadaného uzlu s do všech ostatních uzlů grafu G
- varianty základní úlohy:
 - nejkratší cesty ze všech uzlů do jediného cílového uzlu – stačí řešit základní úlohu v opačně orientovaném grafu G
 - problém jediné nejkratší cesty - není znám žádný algoritmus, který by měl asymptotickou složitost v nejhorším případě lepší než algoritmus pro základní úlohu
 - problém všech nejkratších cest - nalezení nejkratší cesty pro všechny (uspořádané) dvojice uzlů u, v – existuje efektivnější algoritmus než opakovat základní úlohu pro všechny uzly

Nejkratší cesty z jednoho uzlu

Vzdálenost na orientovaných a ohodnocených grafech

- algoritmus vychází z myšlenek BFS

Algoritmus 6.4 Inicializace pro hledání cest

INIT-PATHS(G, s, w)

1 for každý uzel $u \in U$

2 do $d[u] := \infty$

3 $p[u] := \text{NIL}$

4 $d[s] := 0$

Všechny uzly grafu
mají od s nekonečnou vzdálenost
a nemají předchůdce,
jen s je výjimka.

- Rozdíl od BFS - hodnoty $d[u]$ je nutno průběžně upravovat
 - první nalezení uzlu nezaručuje, že se k němu dospělo nejkratší cestou
 - každé následující "znovuobjevení" stejného uzlu u může způsobit snížení dosavadní hodnoty $d[u]$, neboť při určení W -vzdálenosti nehraje roli počet hran minimální cesty, ale součet jejich ohodnocení

Nejkratší cesty z jednoho uzlu

Vzdálenost na orientovaných a ohodnocených grafech

- úpravu vzdálenosti $d[]$ nazveme **relaxací** hrany

Algoritmus 6.5 Relaxace hrany

RELAX(u, v, w)

```
1  if  $d[v] > d[u] + w(u, v)$ 
2      then  $d[v] := d[u] + w(u, v)$ 
3           $p[v] := u$ 
```

Lze-li $d[v]$ zmenšit,
pak to provedeme a také
upravíme předchůdce.

Věta 6.7: Nechť $G = \langle H, U \rangle$ je orientovaný graf s ohodnocením hran $w : H \mapsto \mathbf{R}$, buď $s \in U$ jeho zvolený uzel. Nechť graf G neobsahuje cykly se zápornou w -délkou dosažitelné z uzlu s . Potom po provedení počátečního nastavení procedurou INIT-PATHS(G, s, w) a provedení libovolného počtu relaxací představuje p -podgraf G_p kořenový strom s kořenem s . Navíc, jakmile se pomocí relaxací dosáhne hodnot $d[u] = d_w(s, u)$ pro všechny uzly $u \in U$, je p -podgraf G_p stromem minimálních cest v grafu G do všech uzlů dosažitelných z uzlu s .

Nejkratší cesty z jednoho uzlu

Dijkstrův algoritmus

- ohodnocení hran musí být > 0
- realizace prioritní frontou
- podobná struktura s Jarníkovým-Primovým algoritmem
- taktéž hladový algoritmus

DIJKSTRA(G, s, w)

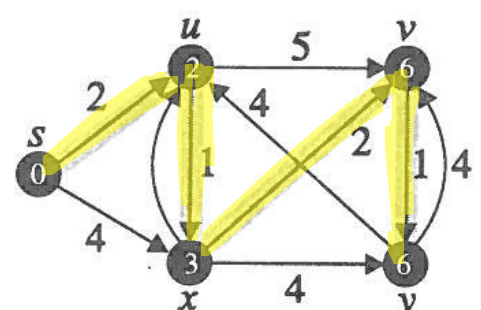
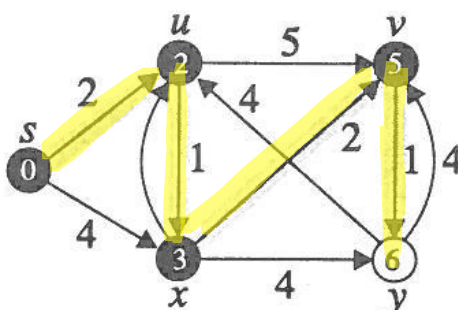
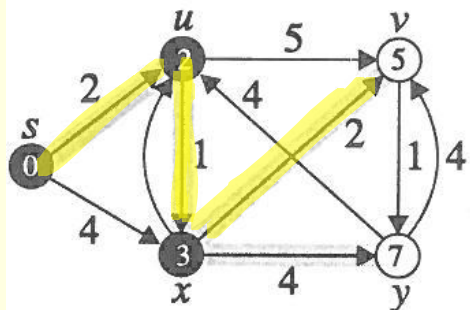
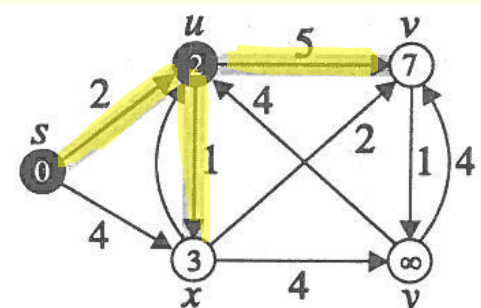
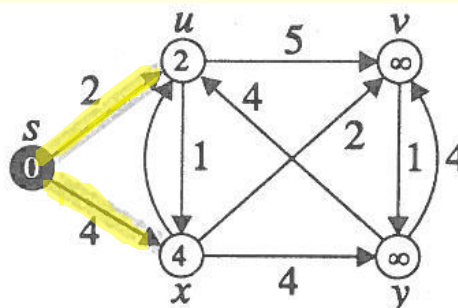
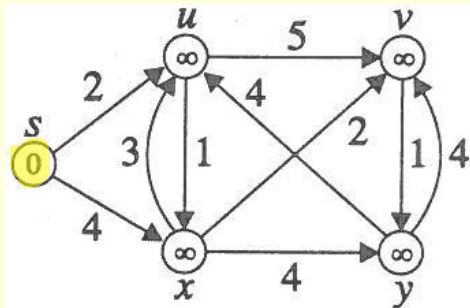
```
1  INIT-PATHS( $G, s$ )
2   $S := \emptyset$ 
3  INIT-QUEUE( $Q$ )
4  for každý uzel  $u \in U$ 
5      do ENQUEUE( $Q, u$ )
6  while not EMPTY( $Q$ )
7      do  $u := \text{EXTRACT-MIN}(Q)$ 
8          $S := S \cup \{u\}$ 
9         for každý uzel  $v \in \text{Adj}[u]$ 
10             do RELAX( $u, v, w$ )
```

Počáteční nastavení $d[u], p[u]$
a zatím žádný uzavřený uzel.

Všechny uzly grafu
se uloží do fronty
seřazené podle $d[u]$
Dokud něco zbývá,
vyber uzel nejbližší k s ,
přeařaď jej mezi uzavřené
a všem jeho následníkům
uprav vzdálenost od s .

Nejkratší cesty z jednoho uzlu

Dijkstrův algoritmus



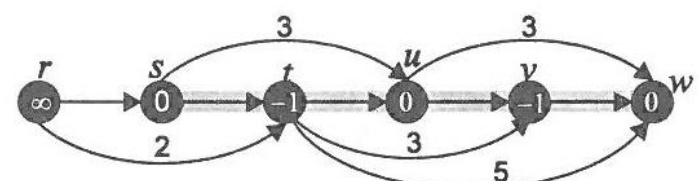
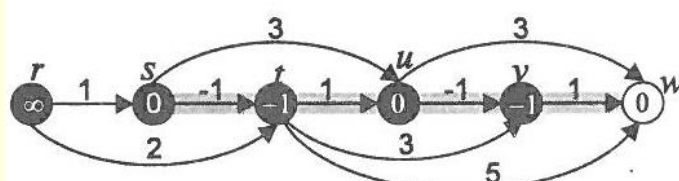
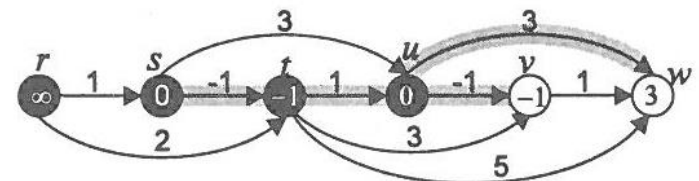
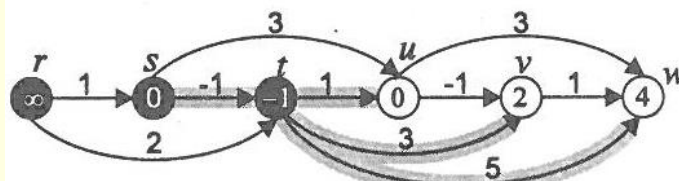
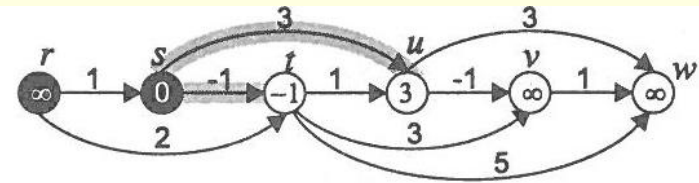
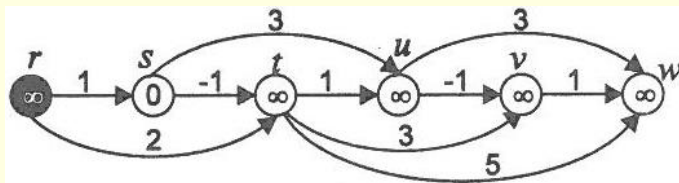
Nejkratší cesta v acyklickém grafu

Algoritmus

Nejkratší cesty pro acyklický graf

DAG-PATHS(G, s, w)

- 1 Topologické uspořádání uzlů grafu G
- 2 INIT-PATHS(G, s)
- 3 **for** každý uzel u v pořadí jeho topologického uspořádání
- 4 **do for** každé $v \in Adj[u]$
- 5 **do** RELAX(u, v, w)



Huffmanovo kódování

Hladový algoritmus (greedy algorithm)

- např. Jarníkův-Primův algoritmus
- dosažení globálně optimálního řešení (např. minimální kostra grafu)
pomocí lokálně optimálních kroků
(výběr uzlu nejbližšího dosavadní kostře)

Huffmanův kód

- využívá průměrné četnosti výskytu symbolů pro kompresi dat
- dle zadaných četností zkonstruujeme binární strom
a z ohodnocení jeho hran vytvoříme kód:
levá hrana bude vždy nula a pravá hrana vždy jednička

Huffmanovo kódování

Huffmanův kód

- mějme četnosti znaků

znak	A	B	C	D	E	F	G	H	I	J
četnost	52	5	18	13	21	4	2	3	35	7

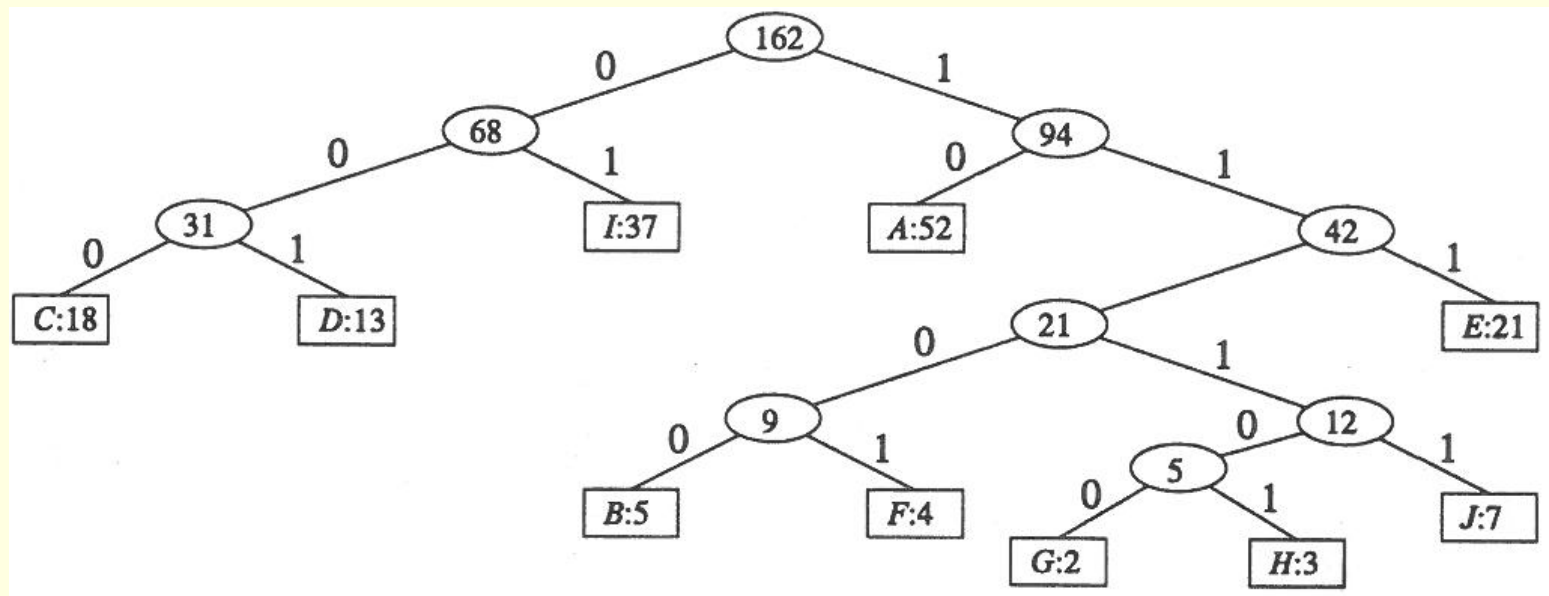
- ze seznamu vyjmeme dva uzly s nejmenší vahou (G:2, H:3) a z nich vytvoříme listy stromu, list s menší vahou umístíme vlevo
- vytvoříme předchůdce těchto dvou listů s vahou danou jejich součtem (X1:5) a zařadíme ho do seznamu uzlů
- postup opakujeme
 - vezmeme F:4 a B:5 a vytvoříme jejich předchůdce (X2:9)
 - vezmeme X1:5 a J:7 a vytvoříme jejich předchůdce (X3:12)
 - vezmeme X2:9 a X3:12 a vytvoříme jejich předchůdce (X4:21)
 - atd...

Huffmanovo kódování

Huffmanův kód

- mějme četnosti znaků

znak	A	B	C	D	E	F	G	H	I	J
četnost	52	5	18	13	21	4	2	3	35	7



znak	A	B	C	D	E	F	G	H	I	J
kód	10	11000	000	001	111	11001	110100	110101	01	11011